

INTERNSHIP REPORT

“ExamGuard AI – AI-Powered Online Examination Management Platform”

BACHELOR OF ENGINEERING

in

INFORMATION TECHNOLOGY

By

Om Chandrakant Deo

Roll No.: T51020

Under Supervision of

Mr. Aadarsh Shinde

Senior Technical Trainer

Rubicon Foundation, Pune

(Duration: From / / 2026 to / / 2026)



DEPARTMENT OF INFORMATION TECHNOLOGY

ZEAL COLLEGE OF ENGINEERING AND RESEARCH, PUNE

Academic Year 2025-26

ZEAL COLLEGE OF ENGINEERING AND RESEARCH, PUNE



CERTIFICATE

This is to certify that the **Internship Report** submitted by **Om Chandrakant Deo (Roll No.: T51020)** is a record of the work carried out by her during the academic year **2025–2026**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Information Technology**. The internship training was successfully completed at **Rubicon Foundation, Pune**

**Internship Coordinator
Department**

**Mr. Aadarsh Shinde
Internship Trainer**

Head of the Department

Place: -

Pune

Date: -

INSERT INTERNSHIP OFFER LETTER HERE, “Remove This line after inserting”

INSERT INTERNSHIP CERTIFICATE HERE, “Remove This line after inserting”

ACKNOWLEDGEMENT

With immense pleasure, I am presenting this report on internship work on **“ExamGuard AI - AI-Powered Online Examination Management Platform”** as a part of the curriculum of T.E. Information Technology at ZEAL COLLEGE OF ENGINEERING AND RESEARCH, PUNE. It gives me proud privilege to complete this report work under the valuable mentorship of Mr. Aadarsh Shinde and Prof. Wrushabh Shirsat.

I am also extremely grateful to Prof. Balaji Chaugule, H.O.D of Information Technology and

Dr. Ajit Kate, Principal Zeal College Of Engineering And Research, Pune for providing all facilities and help for smooth progress of report work.

I would also like to thank all the Staff Members of Information Technology Department, Management, friends, and my family members, who have directly or indirectly guided and helped me for the preparation of this Report and gave me an unending support right from the stage the idea was conceived

Signature

Student Name: Om Chandrakant Deo

Roll No: - T51020

ABSTRACT

THE COMPANY:

The internship was carried out at **Rubicon Foundation**, a technology-driven organization focused on software product development and digital solutions. The company provided a professional and collaborative environment that enabled hands-on learning in full-stack web development and AI-powered application design, bridging academic concepts with real-world implementation.

PROGRAMMERS AND OPPORTUNITIES:

The internship offered valuable opportunities to work with industry-standard technologies such as **React 18, FastAPI, MongoDB, and cloud platforms**. Collaboration with experienced developers provided practical exposure to modern software engineering practices, including **Agile development, code reviews, version control using Git, and cloud deployment pipelines on Render and Vercel**, thereby enhancing both technical and professional competencies.

METHODOLOGY:

The project was developed using an **Agile iterative methodology**, where tasks were divided into structured weekly sprints. Each sprint focused on delivering specific functional modules—such as authentication, examination management, proctoring, and analytics—followed by thorough testing, integration, and review to ensure system reliability and performance.

KEY PARTS OF THE REPORT:

- Company Background Information
- Introduction to Domain — AI-Powered EdTech & Online Examination Systems
- Responsibilities and Contributions During the Internship
- Experience and Technical Skills Acquired
- Detailed Project Report with Screenshots
- Outcomes and Learnings
- Conclusion and Future Scope

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

1st WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED

2nd WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED

3rd WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED

4th WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED

INDEX

Sr. No.	Title	Page no.
1.	Company Background Information	2
2.	Introduction to domain	3
3.	Responsibilities during the internship	4
4.	Experience and skills acquired during internship	5
5	Internship Report & Screenshots	6
6.	Outcomes	31
7.	Conclusion	32
8.	Appendix A: Internship Request Letter	33
9.	Appendix B: Internship Permission Letter	
10.	Appendix C: Daily work sheet	
11.	Appendix D: Assignments during internship	
12.	Appendix E: Internship Certificate	

Learning Objectives/Internship Objectives

Company Background Information

Rubicon Foundation is a technology-driven organization based in Delhi, focused on developing innovative software solutions for the education and enterprise sectors. The company specializes in **full-stack web application development, cloud computing, and AI integration**, delivering scalable and efficient digital solutions to a diverse range of clients.

Founded in 2006, the organization has grown into a team of skilled engineers and designers dedicated to building **secure, reliable, and user-friendly applications**. Rubicon emphasizes modern development practices, leveraging emerging technologies to address real-world challenges across industries.

The internship program at Rubicon Foundation is structured to provide engineering students with **hands-on project experience**, enabling them to work on real-world applications under professional mentorship. This approach helps bridge the gap between academic learning and industry expectations while fostering technical and problem-solving skills.

Introduction to domain

The domain of this internship lies in **AI-Powered EdTech**, with a primary focus on advanced **Online Examination Management Systems** and **Learning Management Systems (LMS)**. In recent years, the rapid expansion of digital education, remote learning environments, and global accessibility to online resources has significantly transformed the traditional education landscape. This transformation has created a growing demand for intelligent platforms that not only deliver educational content but also ensure **secure, scalable, and efficient assessment mechanisms**.

Modern EdTech systems are evolving beyond basic content delivery by integrating **Artificial Intelligence (AI)** to enhance both learning and evaluation processes. These platforms now incorporate features such as **automated proctoring, behavioral analytics, adaptive assessments, and real-time performance tracking**, enabling institutions to maintain academic integrity while providing a personalized learning experience.

A critical aspect of this domain is the development of **secure online examination environments**, where technologies like **behavioral biometrics, anomaly detection, and activity monitoring** are used to identify suspicious patterns and prevent malpractice without relying solely on intrusive methods such as video surveillance. This ensures a balance between **privacy, fairness, and security**.

Additionally, Learning Management Systems are becoming more interactive and learner-centric by offering **structured courses, integrated quizzes, coding assessments, progress tracking, and analytics dashboards**. These features help educators make data-driven decisions while allowing students to monitor their growth and improve their skills effectively.

Overall, AI-powered EdTech platforms aim to create a seamless ecosystem that combines **learning, assessment, monitoring, and analytics** into a unified system, thereby enhancing the quality, accessibility, and credibility of digital education.

Responsibilities during the internship

As an intern on the ExamGuard AI project, the following responsibilities were assigned and fulfilled:

Full-Stack Development

- Designed and implemented the complete full-stack system architecture
- Built 17 Admin portal pages and 12 Student portal pages in React
- Implemented 40+ FastAPI REST API endpoints across 16 route files
- Created 5 MongoDB document models with strict Beanie ODM schemas

Core Feature Engineering

- Engineered the Behavioral Proctoring Engine — delta-based telemetry system with real-time anomaly scoring
- Built the AI Test Generator — curated question bank with 200+ templates across 5+ technical topics
- Integrated the Judge0 code execution proxy endpoint for in-browser code IDE
- Implemented Google OAuth2 SSO with server-side token exchange
- Built the Resend API email service for transactional password reset emails
- Developed the Cold-Start Resilience system for Render free-tier hibernation

Authentication & Security

- Implemented dual-isolated JWT authentication schemes (Student + Admin)
- Applied bcrypt password hashing, timezone-safe datetime comparisons, and strict Pydantic input validation
- Configured CORS, environment variable management, and secret key isolation

DevOps & Deployment

- Dockerized the FastAPI backend and deployed to Render
- Configured Vercel frontend deployment with environment variable injection
- Set up MongoDB Atlas free cluster with network access rules
- Established end-to-end CI/CD via GitHub integration

Architectural Refactoring (10-Phase Audit)

- Led a comprehensive 10-phase codebase audit
- Restructured both frontend and backend to strict domain-driven architecture
- Authored DEVELOPER_GUIDE.md, API_REFERENCE.md, and DEPLOYMENT_GUIDE.md

Experience and skills acquired during internship

Technical Skills Acquired:

- **Full-Stack Development:** Built end-to-end web applications using **React 18** and **FastAPI**, ensuring smooth frontend-backend integration.
- **Asynchronous Programming:** Gained practical experience with **async/await** in FastAPI and **Motor** for efficient, non-blocking operations.
- **Database Design:** Worked with **MongoDB**, designing schemas and implementing models using **Beanie ODM** and **Pydantic**.
- **Security Implementation:** Implemented **JWT-based authentication**, **bcrypt hashing**, and **role-based access control (RBAC)**.
- **API Integration:** Integrated **Judge0 API** for secure code execution and automated evaluation of coding problems.
- **Cloud Deployment:** Deployed applications on **Render** and **Vercel**, managing environment configurations and production setup.

Professional Skills Acquired:

- **Agile Development:** Participated in sprint-based workflows, improving adaptability and planning.
- **Version Control:** Used **Git** and **GitHub** for collaboration, feature branching, and code reviews.
- **Documentation:** Created clear technical documentation and API descriptions.
- **Problem-Solving:** Developed strong debugging and analytical skills while handling real-world issues.
- **Team Collaboration:** Improved communication and teamwork in a professional development environment.

Internship Report & Screenshots

Table of Contents

Sr. No.	Title
1.	Executive Summary
2.	Introduction
3.	Project Objectives
4.	Technology Stack & Justification
5.	System Architecture
6.	Database Design
7.	Key Modules & Features
8.	API Architecture
9.	Security Implementation
10.	Deployment & DevOps
11.	Codebase Refactoring & Architectural Audit
12.	Challenges Faced & Solutions
13.	Key Learnings
14.	Future Scope

1. Executive Summary

ExamGuard AI is a full-stack, enterprise-grade online examination platform built during the internship period. The platform provides a complete end-to-end solution for administering, proctoring, and analyzing academic assessments. It combines real-time behavioral analysis, AI-powered test generation, remote code execution, course management, and a secure multi-role authentication system into one unified application.

The system is designed to serve two distinct user roles — Students and Administrators — each with dedicated portals, protected API layers, and isolated JWT-based authentication schemes. The backend exposes over 40 REST API endpoints across 16 route modules.

The frontend comprises over 29 React page components organized across a clean domain-driven architecture.

The project was successfully deployed to production on Vercel (frontend) and Render (backend), with MongoDB Atlas serving as the cloud database. A comprehensive 10-phase architectural refactoring was also executed to bring the codebase to enterprise production standards.

2. Introduction

The rise of remote learning and online assessments has exposed critical weaknesses in traditional examination systems: lack of proctoring integrity, manual question paper creation, no real-time monitoring, and limited performance analytics. ExamGuard AI was conceived and built to address each of these problems through technology.

The platform is not merely an "online quiz tool." It is a comprehensive academic management system featuring:

- Behavioral Biometrics Tracking — Keystroke patterns, paste events, tab switches, and mouse activity are silently logged during exams to generate an Integrity Risk Score (0–100) for each student.
- Live Admin Monitoring — Administrators can watch student behavior in real-time during examination windows.
- AI-Powered Test Generation — Administrators can generate entire exams (MCQ + coding sections) automatically from a curated question bank spanning multiple topics and difficulty levels.
- Integrated Code IDE — Students can write, run, and test code directly inside the exam interface, powered by the Judge0 remote code execution engine.
- Course Learning Hub — A complete learning management system (LMS) with lessons, quizzes, and enrollment workflows.

3. Project Objectives

The following objectives were defined at the start of the internship and successfully fulfilled:

#	Objective	Status
1	Build a secure multi-role authentication system (Student + Admin)	Complete
2	Implement real-time behavioral proctoring with anomaly scoring	Complete
3	Build an AI-powered exam generator from curated question banks	Complete
4	Integrate Judge0 for in-browser code execution	Complete
5	Build a full Course Management / LMS module	Complete
6	Deploy the entire stack to cloud (Vercel + Render + Atlas)	Complete
7	Perform a 10-phase codebase architectural refactoring	Complete
8	Implement Google OAuth2 for Single Sign-On	Complete
9	Build Admin Analytics dashboard with performance metrics	Complete
10	Create comprehensive developer documentation	Complete

4. Technology Stack & Justification

4.1 Frontend

Technology	Version	Purpose
React	19.x	Core UI framework with Concurrent mode
Vite	7.x	Build tool — faster than CRA, native ES modules
Tailwind CSS	4.x	Utility-first styling, rapid responsive design
React Router DOM	7.x	Client-side SPA routing with nested layouts
Axios	1.x	HTTP client with interceptors and retry logic
Recharts	3.x	Charting library for analytics dashboards

Lucide React	0.5x	Consistent icon set
--------------	------	---------------------

Justification: React 19 was chosen for its mature ecosystem and concurrent rendering capabilities. Vite significantly reduces development cycle time compared to Webpack-based alternatives. Tailwind CSS allows rapid UI prototyping while maintaining strict design consistency.

4.2 Backend

Technology	Version	Purpose
FastAPI	0.135	Async Python REST framework with auto Swagger docs
Python	3.12	Core language — latest stable release
Motor	3.7	Async MongoDB driver for Python
Beanie	2.0	Async ODM/ORM layer over Motor (type-safe models)
Pydantic	2.x	Data validation and serialization
Uvicorn	0.41	ASGI server for production and development
python-jose	3.5	JWT token creation and validation
bcrypt / passlib	—	Secure password hashing
httpx	0.28	Async HTTP client for Google OAuth and Judge0
aiosmtp	3.0	Async email dispatch

Justification: FastAPI was selected for its first-class async/await support, automatic Pydantic validation, and built-in Swagger UI interface for API documentation. Beanie eliminates raw MongoDB query boilerplate while providing full type safety.

4.3 Database

Technology	Purpose
MongoDB Atlas	Cloud NoSQL document database
Motor / Beanie	Async ODM for type-safe queries

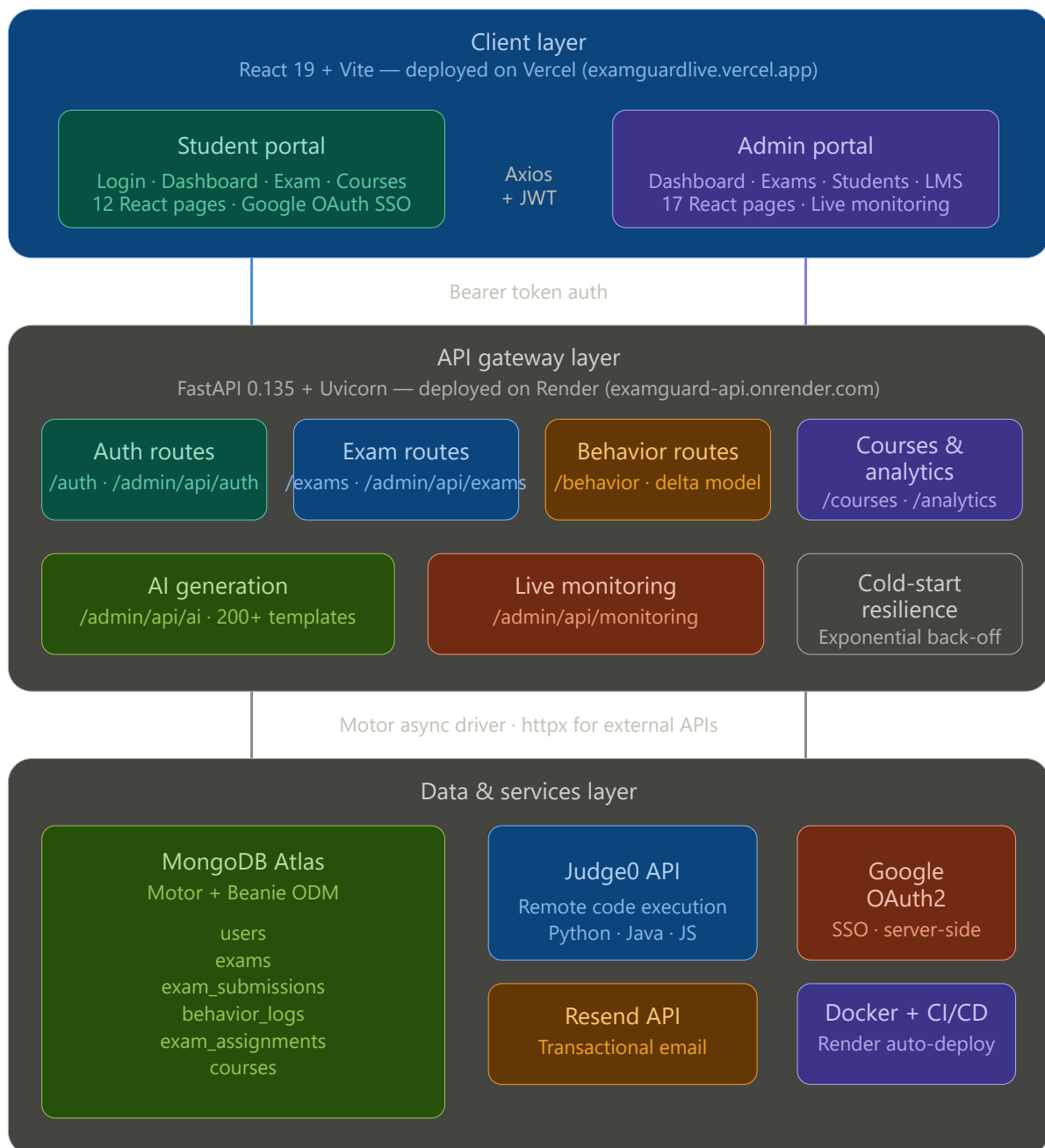
Justification: MongoDB's document model naturally fits the deeply nested exam data structures (sections → questions → options → test cases). Atlas provides a globally distributed free-tier cluster ideal for a production deployment.

4.4 Third-Party Integrations

Service	Purpose
Judge0 API	Remote code compilation and test execution
Resend API	Transactional email (password reset, verification)
Google OAuth2	Single Sign-On for Student portal
Render	Backend PaaS deployment (Docker-based)
Vercel	Frontend deployment with automatic CI/CD

5. System Architecture

5.1 High-Level Architecture



5.2 Frontend Directory Structure

```
frontend/src/  
├── admin/  
│   ├── context/    # AdminAuthContext.jsx (Admin JWT state)  
│   └── pages/      # 17 Admin pages (Dashboard, Exams, Students...)  
├── student/  
│   └── context/    # StudentAuthContext.jsx (Student JWT state)
```

- └─ pages/ # 12 Student pages (Dashboard, ExamPage...)
- └─ components/
 - └─ admin/ # Sidebar.jsx (Admin navigation)
 - └─ student/ # exam/ sub-components (CodeEditor, QuestionPalette...)
 - └─ Modal.jsx
 - └─ ServerWakingOverlay.jsx
- └─ hooks/ # 6 Custom React Hooks (useExamTimer, useProctoringLogger...)
- └─ services/ # API Singletons (apiClient.js, api.js, adminApi.js)
- └─ App.jsx # Root Router + Protected Route wrappers

5.3 Backend Directory Structure

```

backend/app/
├── core/
│   ├── database.py # Motor + Beanie initialization
│   ├── security.py # JWT creation, password hashing (Student)
│   ├── admin_security.py # JWT creation (Admin — isolated keys)
│   ├── logging_config.py # Structured logging setup
│   └── error_handlers.py # Global 422/500 FastAPI handlers
├── models/
│   ├── all_models.py # User, Exam, ExamSubmission, BehaviorLog, ExamAssignment
│   ├── admin_models.py # Admin account document
│   └── course_models.py # Course, Lesson, Quiz, Enrollment documents
├── routes/ # 16 route modules (see Section 8)
├── services/
│   ├── email_service.py # Resend API integration
│   ├── ai_test_generation/
│   │   └── template_repository.py # 200+ question bank
│   └── builder.py # Exam assembly engine
└── utils/
    └── datetime_utils.py # IST timezone helpers
  
```

6. Database Design

The application uses MongoDB with the Beanie ODM. Five core document collections are defined:

6.1 User Collection (users)

```

User {
  _id      : ObjectId
  username  : String (unique, indexed)
  email     : String (unique, indexed)
  password_hash : String
  full_name  : String?
  phone_number : String?
  course     : String?
  college    : String?
  is_active  : Boolean
}
  
```

```

created_at : DateTime (IST)
auth_provider : "local" | "google"
google_id : String?
reset_token : String?
reset_token_expiry : DateTime?
}

```

6.2 Exam Collection (exams)

```

Exam {
  _id : ObjectId
  title : String
  description : String?
  sections : [Section]
  total_marks : Integer
  duration_minutes : Integer
  start_time : DateTime (timezone-aware)
  created_at : DateTime (IST)
}
Section {
  title : String
  questions : [MCQQuestion | CodingQuestion]
}
MCQQuestion {
  id : String
  type : "mcq"
  text : String
  points : Integer
  options : [MCQOption]
}
CodingQuestion {
  id : String
  type : "coding"
  text : String
  points : Integer
  problem_statement : String
  constraints : String
  test_cases : [TestCase]
}

```

6.3 Exam Submission Collection (exam_submissions)

```

ExamSubmission {
  _id : ObjectId
  user_id : String (indexed)
  exam_id : String (indexed)
  exam_title : String
  answers : {section_index: {question_index: answer}}
  status : "IN_PROGRESS" | "COMPLETED" | "GRADED" | "TERMINATED"
  attempt_number : Integer
}

```

```

score      : Float?
mcq_score   : Float?
coding_score : Float?
anomaly_score : Integer (0-100)
risk_level  : "LOW" | "MEDIUM" | "HIGH"
risk_factors : [String]
submitted_at : DateTime
}

```

6.4 Behavior Log Collection (behavior_logs)

```

BehaviorLog {
  _id      : ObjectId
  submission_id : String
  user_id   : String (indexed)
  exam_id   : String (indexed)
  keystroke_count : Integer
  avg_typing_speed : Float (keys/second)
  backspace_ratio : Float
  paste_count  : Integer
  pasted_chars : Integer
  tab_switch_count : Integer
  mouse_click_count : Integer
  time_per_question : {sidx-qidx: milliseconds}
  events        : [{type, timestamp}] (max 500)
  recorded_at   : DateTime
}

```

6.5 Exam Assignment Collection (exam_assignments)

```

ExamAssignment {
  _id      : ObjectId
  exam_id   : String (indexed)
  assigned_students : [user_id_strings]
  max_attempts : Integer
  start_time   : DateTime?
  end_time     : DateTime?
}

```

7. Key Modules & Features

7.1 Student Portal

The Student Portal provides the complete examination experience for end-users.
Pages:

Page	File	Description
Login	LoginPage.jsx	Email/password + Google OAuth SSO

Register	RegisterPage.jsx	Account creation with validation
Google Callback	GoogleCallbackPage.jsx	OAuth token exchange handler
Forgot Password	ForgotPasswordPage.jsx	Email reset request via Resend API
Reset Password	ResetPasswordPage.jsx	Token-validated password update
Dashboard	DashboardPage.jsx	Overview: progress, exams, performance
Waiting Room	WaitingRoomPage.jsx	Pre-exam countdown and instructions
Exam	ExamPage.jsx	Full exam interface (MCQ + Code IDE)
Test Completed	TestCompletedPage.jsx	Score reveal and submission summary
Course List	CourseList.jsx	Browse and enroll in courses
Course View	CourseView.jsx	Lessons, quizzes, progress tracking
Student Profile	StudentProfile.jsx	Profile management and history

Core Student Hooks (in src/hooks/):

Hook	Responsibility
useExamTimer.js	Countdown timer with auto-submit on expiry
useExamNavigation.js	Section/question traversal state management
useQuestionStatus.js	Tracks answered/flagged/visited question states
useProctoringLogger.js	Captures behavioral events and flushes to backend every 15 seconds
useExamSubmission.js	Handles the submit workflow, scoring, and status transitions
useCodeExecution.js	Sends code to Judge0 proxy and handles test results

7.2 Admin Portal

The Admin Portal gives instructors full control over the examination ecosystem.

Pages:

Page	File	Description
Login	Login.jsx	Admin-specific login with isolated JWT
Register	Register.jsx	Create new admin accounts
Dashboard	Dashboard.jsx	System-wide KPI overview
Exam Management	ExamManagement.jsx	List, search, filter all exams
Create Exam	CreateExam.jsx	Manually build exams with sections
Edit Exam	EditExam.jsx	Modify existing exam content
Preview Exam	PreviewExam.jsx	Student-view preview before publishing
Exam Results	ExamResults.jsx	Per-exam score distribution + anomaly flags
AI Test Generator	AITestGenerator.jsx	Automated exam creation from question bank
Students	Students.jsx	Student roster management
Student Progress	StudentProgress.jsx	Individual student learning analytics

Course Management	CourseManagement.jsx	Create and manage course content
Edit Course	EditCourse.jsx	Modify lessons, quizzes, materials
Course Requests	CourseRequests.jsx	Approve/reject enrollment requests
Live Monitoring	LiveMonitoring.jsx	Real-time proctoring dashboard
Analytics	Analytics.jsx	Platform-wide performance analytics
Create Admin	CreateAdmin.jsx	Provision new admin accounts

7.3 AI Test Generator

One of the most technically significant features is the AI-powered exam generation system. How it works:

1. The Admin opens the AITestGenerator page and selects:
 - Topic (Python, Java, JavaScript, DSA, General CS)
 - Difficulty Level (Easy / Medium / Hard)
 - Number of Aptitude MCQs, Technical MCQs, and Coding Questions
2. The request is dispatched to POST /admin/api/ai/generate
3. The backend's template_repository.py module selects questions from over 200+ curated templates spanning:
 - Aptitude MCQs: Speed-Distance, Ratios, Probability, CI/SI, Work-Rate
 - Technical MCQs: Python, Java, JavaScript, DSA, General CS
 - Coding Problems: Two Sum, Valid Parentheses, Fibonacci, LCS, Merge Intervals, N-Queens
4. The assembled exam JSON is returned to the frontend and pre-fills the Create Exam form

Question Bank Scale:

- Aptitude Templates: 22 questions across 3 difficulty tiers
- Technical Templates: 55+ MCQs across 5 topics × 3 difficulties
- Coding Templates: 25+ problems across Python, Java, and General categories

7.4 Behavioral Proctoring Engine

The proctoring system is one of the core differentiators of ExamGuard AI.

Data Captured Per Exam Session:

Signal	Description
keystroke_count	Total keystrokes during exam
avg_typing_speed	Keys per second (weighted average)
backspace_ratio	Backspace : total keystroke ratio
paste_count	Number of paste events (Ctrl+V)
pasted_chars	Characters introduced via paste
tab_switch_count	Number of times student left the exam tab
mouse_click_count	Total mouse clicks

time_per_question	Milliseconds spent on each question
events[]	Raw timeline (up to 500 entries)

Data Flow:

- The useProctoringLogger React hook captures events in the browser continuously.
- Every 15 seconds, a delta payload (only NEW events since last flush) is sent to POST /behavior/log.
- The backend accumulates deltas into a single BehaviorLog document per session.
- On exam submission, the behavior data is analyzed to compute an Integrity Risk Score (0–100).
- Submissions are flagged as LOW, MEDIUM, or HIGH risk with human-readable risk_factors and a risk_explanation.

7.5 Code Execution Engine

The Exam Page includes a full Code Editor IDE for programming-type questions.

Architecture:

Student types code



useCodeExecution hook sends code to:
POST /exams/execute



FastAPI backend proxies to Judge0 API
(<https://ce.judge0.com>)



Judge0 compiles and executes code
Runs against test cases



Returns: { passed, actual, expected, status, error }



UI renders pass/fail per test case

Supported Languages: Any language supported by Judge0, with Language ID passed from the frontend. Default setups include Python 3 (ID: 71), Java (ID: 62), JavaScript (ID: 63).

7.6 Course Management System

The platform includes a complete Learning Management System (LMS):

Student Side:

- Browse available courses with descriptions, lesson counts, and prerequisites
- Submit enrollment requests (pending admin approval)
- Access enrolled courses with structured lessons and video-links
- Track completion with per-lesson checkmarks
- Take module quizzes and see scores

Admin Side:

- Create courses with rich content (markdown descriptions, lesson outlines)
- Manage course modules and their ordering
- View and approve/reject enrollment requests
- Monitor student progress per course through the Analytics dashboard

8. API Architecture

The backend exposes 40+ REST endpoints organized across 16 route module files.

8.1 Route Modules

Module File	Prefix	Description
auth_routes.py	/auth	Student registration, login, OAuth, password reset
exam_routes.py	/exams	Student exam access, start, submit, execute code
behavior_routes.py	/behavior	Proctoring data ingestion (delta model)
course_routes.py	/courses	Course listing, enrollment, progress
course_progress_routes.py	/courses	Lesson completion, quiz submission
student_profile_routes.py	/student/profile	Profile CRUD
admin_auth.py	/admin/api/auth	Admin login, register, me
exam_mgmt.py	/admin/api/exams	Full exam CRUD + assignment
admin_course.py	/admin/api/courses	Course CRUD
admin_enrollment_routes.py	/admin/api	Enrollment approval workflow
student_mgmt.py	/admin/api/students	Student roster management
analytics.py	/admin/api/analytics	Dashboard analytics, exam results
monitoring_routes.py	/admin/api/monitoring	Live proctoring session feeds
admin_progress.py	/admin/api/progress	Student learning progress metrics
ai_generation_routes.py	/admin/api/ai	AI exam generation trigger

8.2 Authentication Design

The system uses two completely separate JWT authentication schemes to prevent privilege escalation:

Attribute	Student JWT	Admin JWT
Secret Key	SECRET_KEY env var	ADMIN_SECRET_KEY env var
Algorithm	HS256	HS256
Expiry	1440 minutes (24h)	1440 minutes (24h)
Guard Function	get_current_user()	get_current_admin()
Header	Authorization: Bearer <token>	Authorization: Bearer <token>

8.3 Cold-Start Resilience

The Render free tier hibernates services after 15 minutes of inactivity. The `ApiClient.js` Axios factory includes:

- Automatic detection of 5xx cold-start errors
- Loading overlay (`ServerWakingOverlay`) shown to users during server wake-up
- Exponential back-off retry logic for failed requests

9. Security Implementation

Feature	Implementation
Password Hashing	bcrypt via passlib library
Student JWT	HS256 signed, 24-hour expiry, verified server-side per request
Admin JWT	Separate HS256 key, completely isolated from student tokens
CORS	Strict allowlist via <code>CORS_ORIGINS</code> environment variable
Google OAuth	httpx async client — token exchanged server-side, user is matched by <code>google_id</code>
Password Reset	Time-limited token (stored hash in DB), compared with <code>secrets.compare_digest</code>
Timezone Safety	All datetime comparisons use timezone-aware objects (UTC/IST) to prevent <code>TypeError</code> crashes
Input Validation	Pydantic V2 schemas validate all request payloads before reaching route handlers
Global Error Handlers	422 Unprocessable Entity and 500 Internal Server Error are caught by <code>error_handlers.py</code>

10. Deployment & DevOps

10.1 Cloud Infrastructure

Layer	Service	Configuration
Database	MongoDB Atlas (Free M0)	Network access: allow all IPs (for Render)
Backend	Render (Docker)	Root dir: backend/, Dockerfile path auto-detected
Frontend	Vercel	Framework: Vite, Root dir: frontend/

10.2 Backend Dockerfile Strategy

The backend is containerized with Docker. Render pulls the image directly from the GitHub repository's Dockerfile. Key environment variables are injected via Render's secret management console:

`MONGODB_URL` = (Atlas connection string)
`SECRET_KEY` = (random 256-bit hex string)

ADMIN_SECRET_KEY = (separate random 256-bit hex string)
CORS_ORIGINS = https://examguardlive.vercel.app
RESEND_API_KEY = (Resend dashboard API key)
FRONTEND_URL = https://examguardlive.vercel.app
GOOGLE_CLIENT_ID = (Google Cloud Console)
GOOGLE_CLIENT_SECRET = (Google Cloud Console)
GOOGLE_REDIRECT_URI = https://examguardlive.vercel.app/auth/google/callback
JUDGE0_API_URL = https://ce.judge0.com

10.3 Frontend Environment (Vercel)

VITE_USER_API_URL = https://examguard-api.onrender.com
VITE_ADMIN_API_URL = https://examguard-api.onrender.com/admin/api

10.4 CI/CD Pipeline

Both platforms provide automatic CI/CD:

- Vercel: Any push to main branch triggers a new frontend build and deployment.
- Render: Any push to main branch triggers a new Docker image build and deployment.

11. Codebase Refactoring & Architectural Audit

A comprehensive 10-phase architectural refactoring was executed as a dedicated workstream to bring the codebase to production enterprise standards.

Phases Completed

Phase	Description	Outcome
Phase 1	Full Codebase Analysis & Mapping	All dead code, orphans, and inconsistencies identified
Phase 2	Remove Unnecessary Files	Deleted 7+ backend diagnostic scripts, orphaned React components
Phase 3	Architectural Restructuring	Flattened nested route dirs; unified services, hooks, components
Phase 4	Code Quality & Consolidation	PascalCase/snake_case enforcement; deduplicated API handlers
Phase 5	Fix Runtime Errors	Patched timezone TypeError crashes; hardened API payloads
Phase 6	Configuration Cleanup	Generated .env.example for both stacks; removed all hardcoded URLs
Phase 7	Documentation	Created DEVELOPER_GUIDE.md, API_REFERENCE.md, DEPLOYMENT_GUIDE.md
Phase 8	System Testing	Verified Auth flows, Exam rendering, Admin panel operations

Phase 9	Performance	Minimized React re-renders; optimized MongoDB query indexes
Phase 10	Final Validation	npm run build completes — 2,500+ modules transformed in 2.53s

Key Structural Changes

Frontend (before → after):

- `src/lib/apiClient.js` → `src/services/apiClient.js`
- `src/admin/components/Sidebar.jsx` → `src/components/admin/Sidebar.jsx`
- `src/admin/services/adminApi.js` → `src/services/adminApi.js`
- `src/student/services/api.js` → `src/services/api.js`
- `src/student/hooks/use*.js` (6 files) → `src/hooks/use*.js`

Backend (before → after):

- `backend/app/db/session.py` → `backend/app/core/database.py`
- `backend/app/routes/routes/` (recursive) → `backend/app/routes/` (flat)
- Production API sanitized of all diagnostic endpoints

12. Challenges Faced & Solutions

Challenge 1: Naive Datetime TypeError Crash

Problem: MongoDB returns timezone-naive datetime objects. Comparing these with timezone-aware Python `datetime.now(UTC)` caused a `TypeError: can't compare offset-naive and offset-aware datetimes` → HTTP 500 crash on the password reset endpoint.

Solution: Implemented a timezone coercion patch in `auth_routes.py`. All datetimes retrieved from MongoDB are explicitly coerced to UTC-aware using `.replace(tzinfo=datetime.timezone.utc)` before any comparison operation.

Challenge 2: SMTP Email Failure on Render

Problem: The initial email service used `aiosmtplib` connecting to `smtp.gmail.com:587`. Render's free-tier infrastructure blocks outbound SMTP on port 587, causing `TimeoutError`.

Solution: Migrated the email service entirely to the Resend API (HTTP-based transactional email), which uses HTTPS port 443 — always available. Replaced the SMTP implementation with a clean `httpx.AsyncClient.post()` call to `https://api.resend.com/emails`.

Challenge 3: Google SSO — Password-less Account Security

Problem: Students who registered via Google OAuth had no `password_hash`. If they later tried the "Forgot Password" flow, there was nothing to reset.

Solution: On Google OAuth registration, a cryptographically random temporary password is generated using `secrets.token_urlsafe(32)` and hashed before being stored. This allows the standard password reset flow to function normally if the user later wants to set a real password.

Challenge 4: Behavioral Proctoring — Submission ID Timing

Problem: The BehaviorLog needed to reference a submission_id, but the submission record is only created when the student starts the exam — the behavior logging begins before that.

Solution: Changed the lookup strategy. During the exam, behavior logs are stored and retrieved by (user_id, exam_id) pair. The submission_id is bound late (updated in the log when the student formally starts). This eliminates the circular dependency.

Challenge 5: Import Resolution After Refactoring

Problem: After physically moving 40+ frontend files across directories (hooks, services, components), all existing import paths broke, causing the Vite build to fail across nearly every component.

Solution: Used a targeted sed command to atomically rewrite all import paths across every affected file in one pass. The npm run build was used as the ground-truth validator after each batch of path corrections.

13. Key Learnings

Domain	Learning
FastAPI	Async route handlers, Beanie ODM, Pydantic V2 schema design, JWT middleware
React Architecture	Domain-driven component design, Custom hooks as business logic containers, Context API for auth state
MongoDB	Document schema design for deeply nested data, compound index optimization, Beanie ODM patterns
Security	JWT isolation between roles, bcrypt hashing, OAuth2 PKCE flow, timezone-safe datetime comparisons
DevOps	Docker containerization, Vercel/Render CI/CD pipelines, Atlas network configuration
API Design	REST convention adherence, consistent error response structure, HTTP status code semantics
Problem Solving	Debugging production SMTP blocks, resolving proctoring data race conditions, fixing import paths at scale
Engineering	How to systematically audit, refactor, and document a large codebase

14. Future Scope

Feature	Priority	Description
ML Anomaly Detection	High	Train a scikit-learn classifier on historical behavioral logs to replace the rule-based anomaly scoring
Video Proctoring	High	Integrate WebRTC + MediaRecorder API for optional webcam snapshots during exam sessions
Plagiarism Engine	Medium	Run MOSS (Measure of Software Similarity) on submitted code to detect copy-paste plagiarism
Email Notifications	Medium	Send automated score reports and exam reminders to students

Mobile App	Medium	Build a React Native app for students to access the platform on mobile
Question Bank Editor	Low	Admin UI to add/edit questions directly into the <code>template_repository.py</code> question bank
WebSocket Live Monitoring	Low	Replace HTTP polling with WebSocket for truly live proctoring feeds
Multi-Tenant Support	Low	Allow multiple institutions to run isolated instances on the same backend

Candidate Registration:

examguardlive.vercel.app/register

Candidate Registration

Create your account to begin the assessment process.

Username
Choose a unique username

Email Address
name@example.com

Password
Create a strong password
Must be at least 8 characters long.

Create Account

or

Continue with Google

Already have an account? [Sign In](#)

By registering, you agree to our [Terms of Service](#) and [Privacy Policy](#) regarding biometric data collection.

ExamGuard

Join thousands of professionals who have validated their skills through our secure, unbiased assessment platform.

- 1 Create Profile**
Set up your secure candidate identity.
- 2 Verify Email**
Confirm your identity securely.
- 3 Take Assessment**
Prove your skills securely.

Candidate Login:

examguardlive.vercel.app/login

Candidate Portal

Sign in to access your assigned assessments.

Username / Candidate ID
Enter your ID

Password
Enter your password
[Forgot password?](#)

Sign In securely

or

Continue with Google

New Candidate? [Create Application Account](#)

ExamGuard

Welcome to our secure assessment portal. We leverage advanced behavioral biometrics to ensure fair and integrity-driven evaluations for all candidates.

AI Verified
Real-time Analysis

Secure
Bank-grade Encryption

© 2026 ExamGuard Global. All rights reserved.
Crafted by Om Chandrakant Deo

Admin Login:

examguardlive.vercel.app/admin/login

ExamGuard

Secure Administration Portal. Manage assessments, monitor integrity, and oversee candidate performance with advanced analytics.

Admin Access
Restricted Area

Admin Portal
Sign in to access administrative controls.

Username
Enter admin username

Password
Enter secure password

Sign In

[Don't have an account? Create one](#)

© 2026 ExamGuard Global. All rights reserved.
Crafted by Om Chandrakant Deo

Admin Dashboard:

examguardlive.vercel.app/admin/dashboard

ExamGuard Admin

- Dashboard
- Exam Management
- Courses
- Enrollments
- Students
- Live Radar
- Analytics
- Student Progress
- Create Admin

Admin Dashboard
Manage system overview, exams, and student records.

TOTAL EXAMS
2
All assessments

TOTAL STUDENTS
28
Registered candidates

SUBMISSIONS
16
Total completed

AVG SCORE
14
Across all exams

Recent Exams [View All →](#)

- Basic Demo** Completed
This is a demo exam to test portal
31/3/2026 10 mins [Details](#)
- Full Stack Assessment (Final)** Completed
Comprehensive exam covering Aptitude (30), Technical (30), and Coding (2).
15/3/2026 180 mins [Details](#)

Recent Activity

- Full Stack Assessment (Final)** COMPLETED
User: 69bc51b8b5a813bdc990604 Score: 4 25/3/2026
- Backend Test Exam 1774371954** COMPLETED
User: 69c2c47431da48ac98891253 Score: 20 24/3/2026
- Backend Test Exam 1774371954** COMPLETED
User: 69c2c47431da48ac98891253 Score: 20 24/3/2026
- Backend Test Exam 1774371838** COMPLETED
User: 69c2c400368c68bdfd847fee Score: 20 24/3/2026
- Backend Test Exam 1774371838** COMPLETED
User: 69c2c400368c68bdfd847fee Score: 20 24/3/2026

admin
admin@example.com
[Sign Out](#)

© 2026 ExamGuard Global - Crafted by Om Chandrakant Deo

Exam Management:

ExamGuard Admin

Dashboard

Exam Management

Courses

Enrollments

Students

Live Radar

Analytics

Student Progress

Create Admin

admin

admin@example.com

Sign Out

© 2026 ExamGuard Global - Crafted by Om Chandrakant Deo

Exam Management

Create and manage your examination system

Create with AI

Create New Exam

TOTAL EXAMS

2

ACTIVE EXAMS

0

TOTAL MARKS

90

AVG DURATION

95m

Full Stack Assessment (Final)

Past

Comprehensive exam covering Aptitude (30), Technical (30), and Coding (2).

Sun, Mar 15, 2026

180 minutes

80 marks

3 sections

Basic Demo

Past

This is a demo exam to test portal

Tue, Mar 31, 2026

10 minutes

10 marks

1 sections

Student Management:

ExamGuard Admin

Dashboard

Exam Management

Courses

Enrollments

Students

Live Radar

Analytics

Student Progress

Create Admin

admin

admin@example.com

Sign Out

© 2026 ExamGuard Global - Crafted by Om Chandrakant Deo

Student Directory

Manage accounts and analyze candidate performance

Add Student

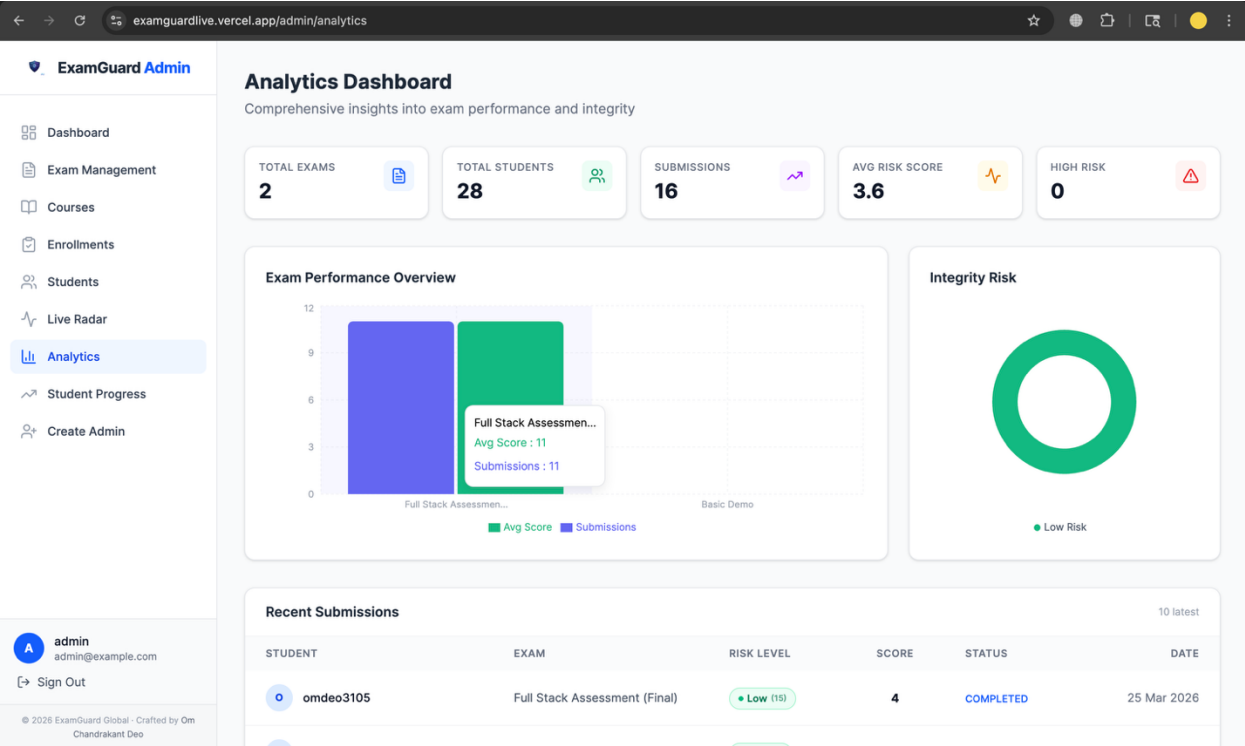
TOTAL REGISTERED

28

Search by student name or email...

CANDIDATE	CONTACT INFO	ACCOUNT STATUS	REGISTRATION DATE
S student_2925a1 ID: 69b1b28f...	✉ student_2925a1@example.com	Active	11 Mar 2026
T testuser ID: 69b1e370...	✉ test@example.com	Active	11 Mar 2026
T testuser_test ID: 69b6616f...	✉ test@test.com	Active	15 Mar 2026
T testuser_e774817b ID: 69b679f7...	✉ test_e774817b@example.com	Active	15 Mar 2026
S student_test ID: 69b6c93a...	✉ student@test.com	Active	15 Mar 2026
O Om ID: 69b6e023...	✉ om@gmail.com	Active	15 Mar 2026
O Om3 ID: 69b8fc17...	✉ om3@gmail.com	Active	17 Mar 2026
O om4 ID: 69b8fdb5...	✉ om4@gmail.com	Active	17 Mar 2026

Analytics Dashboard:



Create New Admin:

ExamGuard Admin

Create Account
Register for administrative access.

Full Name
Enter your full name

Username
Choose a username

Email Address
admin@organization.com

Password

Create Account →

Already have an account? [Sign in here](#)

ExamGuard
Join the Secure Administration Portal. Manage assessments and monitor integrity.

Footer: © 2026 ExamGuard Global. All rights reserved. Crafted by Om Chandrakant Deo

Candidate Dashboard:

← → ↺ examguardlive.vercel.app/dashboard

ExamGuard Global

Dashboard

Learning Hub

omdeo315
Student Account

o

Sign Out

Candidate Dashboard

Manage your assessments and view your performance reports.

LEARNING PROGRESS

Enrolled

1 / 2 courses

Explore courses in the Learning Hub

EXAM PERFORMANCE

Score

0 completed

1 assigned

1 exam available to attempt

PENDING EXAMS

View All

1

No assessments scheduled for this week.

My Courses — Continue Learning

View All →

Python Ultimate Course

Learn Python from scratch in this complete 13-hour course designed for beginners who want to build real programming...
46 lessons • 8 modules

Continue Learning →

Available Now

Upcoming

Completed

Refresh List

B

Basic Demo

Start Exam

← → ↺ examguardlive.vercel.app/dashboard

ExamGuard Global

Dashboard

Learning Hub

omdeo315
Student Account

o

Sign Out

My Courses — Continue Learning

View All →

Python Ultimate Course

Learn Python from scratch in this complete 13-hour course designed for beginners who want to build real programming...
46 lessons • 8 modules

Continue Learning →

Available Now

Upcoming

Completed

Refresh List

F

Full Stack Assessment (Final)

Comprehensive exam covering Aptitude (30), Technical (30), and Coding (2).
📅 15 Mar 2026, 08:14 pm ⌚ 180 Mins 📊 80 Marks

Start Exam

Attempts: 0 / 1

B

Basic Demo

This is a demo exam to test portal
📅 31 Mar 2026, 04:22 pm ⌚ 10 Mins 📊 10 Marks

Start Exam

Attempts: 0 / 1

26

Exam Flow:

examguardlive.vercel.app/exam/69cba790bed4a8726bf80bb/waiting-room

ExamGuard

Exam ID: 69cba790bed4a8726bf80bb

EXAM IS LIVE

Basic Demo

This is a demo exam to test portal

00

DAYS

00

HOURS

00

MINUTES

00

SECONDS

Enter Exam Now →

IMPORTANT INSTRUCTIONS

✓ Don't switch browser tabs.

✓ Microphone must be on.

✓ Keep your camera enabled.

✓ Stable internet required.

Session ID: 8f92-a1b2-c3d4-e5f6 • Protected by ExamGuard AI

examguardlive.vercel.app/exam/69b6c5ba15e72d1495114749

Full Stack Assessment (Final)

02:59:51

LEGEND

Not Visited

Not Answered

Answered

Marked for Review

Answered & Marked

ANSWERED

0

NOT ANSWERED

1

MARKED

0

NOT VISITED

61

APTITUDE MCQ

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

TECHNICAL MCQ

1

2

3

4

5

APTITUDE MCQ / QUESTION 1

1 Mark

Sample Aptitude Question 1: What is the output of standard logic?

Option A (Correct)

Option B

Option C

Option D

Clear Response

Mark for Review & Next

Save & Next

Finish Test

27

Full Stack Assessment (Final)

02:58:35

ARTHOGEOMETRY

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25
26	27	28	29	30

TECHNICAL MCQ

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25
26	27	28	29	30

CODING

1	2
---	---

CODING / QUESTION 1

10 Marks

Sum of Two Numbers

Write a program that takes two integers as input (separated by a space or newline) and prints their sum.

CONSTRAINTS:

1 <= a, b <= 1000

Code Editor

Python (3.8.1)

```
a, b = map(int, input().split())
print(a + b)
```

Full Stack Assessment (Final)

02:58:28

ARTHOGEOMETRY

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25
26	27	28	29	30

TECHNICAL MCQ

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25
26	27	28	29	30

CODING

1	2
---	---

Run Code

Test Results:

Test Case 1

Input: 2 3
Expected: 5
Actual: 5

✓ Passed

Test Case 2

Input: 10 20
Expected: 30
Actual: 30

✓ Passed

Test Case 3

Input: 100 200
Expected: 300
Actual: 300

✓ Passed

Clear Response

Mark for Review & Next

Save & Next

Finish Test



Tab Switch Detected!

Switching tabs or leaving the exam window is a violation of exam integrity rules.

⚠ Violation 2 of 3 — 1 remaining before auto-submit

[I Understand — Return to Exam](#)

examguardlive.vercel.app/exam/69b6c5ba15e72d1495114749



Finish Exam?

Are you sure you want to submit your exam?

You cannot change your answers after submission.

[Cancel](#) [Confirm Submit](#)



Test Completed Successfully

Exam

Full Stack Assessment (Final)

Submitted At

April 2, 2026 at 01:54 AM



Your responses have been successfully submitted.

You may now safely return to the dashboard.

[Return to Dashboard](#)

Outcomes

The following outcomes were successfully achieved by the end of the internship:

- A **fully functional dual-portal web application** (Admin and Student) was developed, deployed, and made accessible through public URLs, demonstrating end-to-end system integration.
- Implementation of a complete **examination workflow** supporting both MCQ and coding formats, including automated evaluation, result generation, and submission tracking.
- Development of a **real-time proctoring system** capable of detecting tab-switching, clipboard actions, and behavioral anomalies, with all violations logged and available for administrative review.
- A comprehensive **Learning Management System (LMS)** enabling course creation, content delivery (videos, notes, quizzes), student enrollment, progress tracking, and certificate generation.
- Creation of **interactive analytics dashboards** for administrators, providing insights into student performance, exam statistics, and integrity risk scores with efficient and responsive data visualization.
- Successful integration of external services such as **Judge0 for secure code execution**, ensuring accurate and scalable evaluation of coding assessments.
- Deployment of the system using **cloud platforms (Render and Vercel)**, ensuring accessibility, reliability, and real-world usability of the application.
- Significant personal development in **full-stack development, asynchronous programming, API integration, system design, and cloud deployment practices**, along with improved problem-solving and debugging skills.

Conclusion

ExamGuard AI successfully demonstrates the development of a production-grade, enterprise-scale full-stack web application from concept to deployment. The project integrates a diverse set of modern technologies — React 19, FastAPI, MongoDB, Docker, Google OAuth, Judge0, and Resend — into a cohesive, well-architected system.

The internship provided deep, practical experience in:

- Designing clean REST APIs that are secure, consistent, and well-documented
- Managing complex state in large React applications through custom hooks and context providers
- Operating cloud infrastructure (Vercel + Render + MongoDB Atlas)
- Performing systematic architectural audits and large-scale codebase refactoring
- Debugging production issues including SMTP blocks, timezone crashes, and data race conditions

The resulting platform is fully functional, deployed to production, and maintained under version control with professional documentation that enables any new developer to understand, extend, and deploy the system independently.

© 2026 Om Chandrakant Deo — ExamGuard AI Internship Project Report Live Application: <https://examguardlive.vercel.app> GitHub Repository: <https://github.com/om3105/ExamGuard-AI>

Appendix C

Daily Work Sheet

Duration: (in Hours)

Day No.	Activity Planned	Activity Completed Status	Student Signature	Mentor Signature
Day 1				
Day 2				
Day 3				
Day 4				
Day 5				
Day 6				
Day 7				
Day 8				
Day 9				
Day 10				
Day 11				
Day 12				
Day 13				
Day 14				
Day 15				
Day 16				
Day 17				
Day 18				
Day 19				
Day 20				
Day 21				
Day 22				
Day 23				
Day 24				

INTERNSHIP – DAILY WORKING REPORT SHEET

- **Lab Session Details**

Date	Day	Time (From – To)

- **Project Information**

Title of Project:	ExamGuard AI — AI-Powered Online Examination Management Platform
Objective Name	
Title of Work Performing:	

- **Project Parameters Involved**

- ☐ Frontend ☐ Backend ☐ Database ☐ Machine Learning / AI ☐ API Integration
☐ Simulation / Hardware ☐ Testing / Optimization

- **Student Information**

Roll Number	Student Name	Signature

- **Work Done During Lab Session / Work Completion**

(Describe the exact work performed during this lab duration)

- **Remarks by Verifying Faculty (Satisfactory / Not-Satisfactory)**

Appendix D

Assignments during internship

Sr. No.	Name of Assignment/Activity	Status (Completed/NC)	Remark